Named Axes as a Systems Discipline: Hybrid Compile-Time and Runtime Coordination for Tensor Programs

[TODO: author block — fill at submission time]
authors@example.com

Abstract

Tensor programs routinely fail on *axis-bug* classes that current type systems cannot catch: silent broadcasts across mis-named dimensions, transposed contractions, and shape-compatible but semantically incorrect reductions. Existing named-axis libraries address this either through runtime string-based notation (`einops`, `einx`) or language-level named tensors with fully runtime checks (`Haliax`, `Penzai`, `xarray`). We argue these approaches under-use the host language's static type system, and present a hybrid design that encodes axis information across the compile-time / runtime boundary. The contributions are: (i) a `_tex` user-defined literal that parses a LaTeX-math subset (e.g. `R"(c_{ij} = a_i b_j)"_tex`) into a named-axis expression AST whose structural indices the type system can reason about, letting the same formula compose with the static-axis machinery at no boundary cost; (ii) a hybrid *NTTP* × *DynamicShape* axis system that statically rejects axis-name mismatches when shapes are known and falls back to runtime guards otherwise, sharing one kernel implementation across both paths; (iii) a *Hexagonal-lite* kernel-backend port that demonstrates substrate substitutability across a reference implementation, an Eigen-based substrate, and a WebGPU substrate (Dawn) behind a single port boundary. **[TODO: 1-sentence headline empirical result once the B-stage bench harness lands — e.g., "We catch 87% of the axis-bug benchmark suite at compile time with zero runtime cost; [BENCH: X]-fold speedup on the bundle-adjustment case study."]**

1 Introduction

Consider a researcher writing a multi-head attention block. They compose two linear projections, a softmax, and a matmul. The shapes type-check at every step — every intermediate has the right number of dimensions — yet the network silently produces incorrect outputs at training time, because two axes were transposed five lines up. The type system saw a (B, H, T, D) tensor where the writer meant (B, T, H, D); the names lived only in a comment.

This failure mode is endemic. **[TODO: cite a concrete bug-hunting study or anecdote — ideally one of: PyTorch named-tensors postmortem, Haliax design doc on motivation, our own M1 landscape report [12].]** Existing libraries respond along two axes:

- **String-based runtime notation** (`einops` [7], `einx` [2]): expressive (Einstein-style formulas are the source of truth) but defers all checking to runtime and forfeits the host language's type system.
- **Language-level named tensors with runtime checks** (`Haliax` [8], `Penzai` [4], `xarray` [5], deprecated `torch.named_tensor` [6]): axis names are first-class values, but verification still happens at runtime.

The thesis. A named-axis library should be able to refute the attention-block bug *at compile time, when the shapes are known*, while still permitting the runtime-shape case that batch processing demands. The test of a design is not whether axis names are present but whether they participate in the type system.

Contributions. This paper presents the `tensor` library, organised around three design moves:

1. **The `_tex` LaTeX bridge** (Section 3): a user-defined literal that parses a LaTeX-math subset (`R"(c_{ij} = a_i b_j)"_tex`) into a named-axis `Expression` AST. The AST's index structure — the names attached to each `IndexedVar` and the bound index on each `Sum` — is what the type system reasons about, so the same formula composes with the static-axis machinery of contribution (ii) at no boundary cost.
2. **Hybrid axis system** (Section 4): non-type template parameters carry compile-time axis sizes; `DynamicShape` carries the runtime-determined cases; the two compose without bifurcating the kernel API.
3. **Hexagonal-lite kernel-backend port** (Section 5): a single port boundary admits three substrates (reference, Eigen, WebGPU-via-Dawn [3, 10]) without leaking substrate concerns into the named-axis layer.

Roadmap. Section 2 surveys the axis-bug taxonomy and the prior-art landscape. Sections 3 to 5 present the three design moves. Section 6 reports the static-catch benchmark and a bundle-adjustment case study across three backends. Section 7 positions against contemporaneous work. Section 8 discusses limitations.

2 Background & Motivation

2.1 The axis-bug taxonomy

[TODO: taxonomy table with 4–6 axis-bug classes, each with a 1-line `torch`/`numpy` reproduction. Cross-reference against the M1 landscape report.]

2.2 Landscape

[TODO: landscape table; axes = "where axis names live" $\times$ "when they are checked". Place einops, einx, Haliax, Penzai, xarray, named PyTorch, tinygrad, and our system in this 2-D grid. Result: our system uniquely occupies the (type-parameter, compile-time-when-known) cell. Compress to <1 column of body text.]

2.3 What remains open

The landscape leaves an explicit gap: *compile-time axis checking that does not abandon the runtime-shape case*. The static approaches in the literature require either fully static shapes (no batch flexibility) or full type erasure at the kernel boundary. Our design target is to recover the latter without giving up the former.

3 Design 1: The `_tex` LaTeX Bridge

3.1 Interface

The user writes the formula the way the corresponding paper would, in a C++ raw-string literal suffixed `_tex`, and feeds it through an Evaluator bound to named tensors:

```cpp
// Bind names -> tensors, then evaluate the formula.
tex::Evaluator<double> ev;
ev.bind("a", DynamicTensor{Shape{Axis{"i", 5}}, /*data*/});
ev.bind("b", DynamicTensor{Shape{Axis{"j", 5}}, /*data*/});

auto c = ev.evaluate(R"(c_{ij} = a_i b_j)"_tex);
// c has named axes (i, j); c[i,j] == a[i] * b[j].
```

The literal `R"(c_{ij} = a_i b_j)"_tex` parses on construction into an `Expression` AST. The `tex::Evaluator` walks the AST against name-to-tensor bindings, asserting that each `IndexedVar`'s subscript list matches its bound tensor's axis labels in order. For example, `R"(a_{ji})"_tex` bound against a tensor whose axes are declared (i, j) is rejected by the Evaluator's subscript-order check.

3.2 Grammar

The accepted grammar is the LaTeX-math subset useful for naming and combining axis-indexed quantities (informal BNF in priority order):

```
equation    := expression ('=' expression)?
expression  := term (('+' | '-') term)*
term        := factor (('*' | '\cdot' | juxt) factor)*
factor      := ('-' factor) | unary
unary       := atom (('/' atom)*)
atom        := '(' expression ')'
             | '\sum' '_' subscript group
             | name subscript?
subscript   := '_' (letter | '{' letters '}')
group       := '{' expression '}' | atom
```

Whitespace is ignored; juxtaposition is implicit multiplication. The parser is a hand-written recursive descent; the grammar is small enough that the implementation fits in a single header (`include/tensor/tex/parser.hpp`).

The choice of LaTeX over an einops-style mini-language is deliberate. The subset that matters — subscripted variables, Einstein sums (`R"(\sum_i {a_i b_i})"_tex`), and equations that name a result — is the notation tensor-program papers and textbooks already use. The literal is also *readable as LaTeX*, so the same text that drives evaluation can be re-emitted into the program's typeset output: a Jupyter cell prints back the original formula, closing the loop from notation to running code to rendered documentation.

3.3 Composability with the Static-Axis System

The system-level claim of this section is not about *when* the parse fires — it fires at literal construction today, which is runtime — but about *what shape* it produces. The parsed AST's indices are the same structural objects the static-axis machinery of Section 4 reasons about. The Evaluator's binding step is a type-level operation when the bound tensor's axes are NTTP-encoded, and a runtime check when they are carried in a `DynamicShape`. The Evaluator does not branch on which world it is in; one code path services both.

[TODO: one-paragraph walk-through of one literal (`R"(c_{ij} = a_i b_j)"_tex`) feeding three scenarios: (a) two NTTP-shaped tensors — the bind step is compile-time-checked; (b) one NTTP, one `DynamicShape` — names checked statically, sizes at `bind` time; (c) two `DynamicShape` — both checks at `bind` time. The formula text is unchanged across all three.]

3.4 Scope and the Scheduled `consteval` Lift

The parser is presently runtime, executed at the literal's construction site. The natural `consteval` formulation is blocked by C++20's prohibition on `std::unique_ptr` escaping a constant-evaluated context (the current AST is a `unique_ptr`-of-variant tree). ADR-0005 [9] schedules a refactor that replaces the `unique_ptr` representation with a fixed-size arena-of-variants whose lifetime is the literal's; that representation does escape constant evaluation and lifts the entire parse into the compiler.

Three properties of the current design are unchanged by the lift, and are this section's contribution:

1. the parser's output type (`Expression`) is identical in both worlds;
2. the Evaluator's binding-and-check pass is identical;
3. the parse cost is paid once per literal, not once per call — today at static-initialiser time, after the lift at compile time.

Grammar-wise, the deliberate non-goals are: no ellipsis, no broadcast operators, no rank-polymorphic patterns, no run-time–constructed strings (the literal arrives as `char const*`). Cases the grammar cannot express are carried by the hybrid system of Section 4.

4 Design 2: Hybrid NTTP × DynamicShape

4.1 The two paths

A named tensor carries its axis information in one of two ways:

Compile-time path axis names *and* sizes are encoded as non-type template parameters. A literal expecting axes (i, j) — e.g. `R"(c_{ij} = a_i b_j)"_tex` — bound through the Evaluator against a tensor whose NTTP axes are (i, k) is refuted at template instantiation; the diagnostic names the offending axis.

Runtime path axis names are encoded as template parameters (compile-time) but sizes live in a `DynamicShape` value (runtime). The compiler refutes axis-*name* mismatches; size mismatches surface as a `ShapeError` at the kernel call site.

4.2 Promotion and demotion

[TODO: promotion/demotion rules with a one-page minimum example (static-shaped activations flowing into a runtime-batched op). Include the C++ snippet showing what the user writes vs what the compiler instantiates.]

4.3 Kernel boundary

The contract of a kernel is a function template parameterised by the axis structure (compile-time) accepting `DynamicShape` runtime sizes when needed. *The same kernel* services both paths; the runtime-path overhead is one bounds check per call, not a separate kernel implementation.

5 Design 3: Hexagonal-lite Kernel Backend

The named-axis layer should not know whether contractions are computed by hand-rolled loops, an Eigen template instantiation, or a WGSL kernel dispatched to a GPU. *Hexagonal-lite* is a constrained form of the Hexagonal architecture [11]: a single port (`KernelBackend`) with three concrete adapters.

5.1 The port

[TODO: one-page C++ listing of the `KernelBackend` concept (in the C++20 `concept` sense) — the minimal operations a substrate must provide, with brief commentary on each requirement.]

5.2 Substrates

Reference portable naive nested-loop implementation; defines the semantics that the other substrates must match.

Eigen BLAS-routed contractions for CPU performance baseline; loop fusion via Eigen's expression templates.

WebGPU via Dawn [3, 10] GPU substrate compiled to WGSL; vcpkg-vendored Dawn to keep substrate provisioning reproducible in CI.

The choice of substrate is a CMake variable (`-DTENSOR_KERNEL_BACKEND={reference,eigen,webgpu}`) — the named-axis layer is unchanged across all three.

6 Evaluation

6.1 Static-catch benchmark

[BENCH: Static-catch benchmark suite: ∼50 known-buggy programs spanning the taxonomy in Section 2.1. Score each entry as {compile-time, runtime, silent} for each of: `tensor` (ours), `einops`, `einx`, `Haliax`, `Penzai`. Headline table = library × {caught at CT, caught at RT, silent} with percentages.]

6.2 Bundle-adjustment case study

[BENCH: 50-view × 200-landmark bundle-adjustment scene from a public dataset (e.g., BAL). Three backends (reference / Eigen / WebGPU). Metrics: time-to-residual, time-to-gradient, peak memory. Comparison baselines: `einops.rearrange`-based Jacobian assembly, `haliax.NamedArray` on JAX backend.]

[TODO: at minimum a single table with rows = backend, columns = metric. Append a small figure showing scaling with view count if space permits.]

6.3 Threats to validity

[TODO: honest discussion: benchmark size, single-author bias on the axis-bug suite, hardware/dataset variability for the BA case study.]

7 Related Work

[TODO: 1-page synthesis from the M1 study [12]. Each paragraph closes with what our system carries over and what it diverges on. Notable: Chiang & Rush [1] give the notation; we give a type-system implementation of it.]

8 Discussion and Limitations

When to reach for the compile-time path. **[TODO: small decision-aid: shapes known at site of use → CT path; batch size variable but axes named at compile time → hybrid; fully dynamic (e.g., loaded from disk) → runtime path with upgrade-to-CT after the first call.]**

Limitations. **[TODO: honest enumeration: (L1) UDL grammar deliberately small (no ellipsis, no rank polymorphism); (L2) compile-time error messages are template-instantiation-deep — we provide diagnostic helpers but C++'s default error rendering is unhelpful; (L3) WebGPU substrate is in early adoption; portability beyond Dawn-supported platforms is currently constrained.]**

Future work. The primary scheduled lift is moving the `_tex` parser from runtime (literal-construction time) to `consteval` (compile time) by replacing the `unique_ptr`-of-variant AST with a fixed-size arena representation that escapes constant evaluation (Section 3.4, ADR-0005 [9]); the output type and Evaluator pass are unchanged, so this is a mechanical refactor whose payoff is moving parse cost from program startup to compile time. Beyond that: extending the grammar towards rank-polymorphic patterns without losing the structural-index property; auto-tuned substrate selection based on shape and target; integration with Triton-class kernel languages at the Hexagonal-lite adapter layer.

9 Conclusion

Named-axis libraries should treat the host language's type system as load-bearing infrastructure, not as a place to attach metadata. The `tensor` library demonstrates that the static and dynamic worlds compose: a LaTeX-bridging `_tex` UDL whose AST shape is the same object the type system reasons about, a hybrid NTTP × DynamicShape axis system that shares one kernel across both paths, and a Hexagonal-lite kernel backend that demonstrates substrate substitutability across three concrete adapters. **[TODO: one-sentence summary of the headline empirical result once [BENCH:] numbers land.]**

References

[1] David Chiang and Alexander M. Rush. Named tensor notation. In *Transactions on Machine Learning Research*, 2024. Formalises named-axis notation as a typed calculus.

[2] einx contributors. einx: Universal tensor operations using einstein notation. `https://github.com/fferflo/einx`, 2023–2026.

[3] Google Chrome. Dawn: A WebGPU implementation. `https://dawn.googlesource.com/dawn`, 2020–2026.

[4] Google DeepMind and contributors. Penzai: A jax research toolkit with named tensors. `https://github.com/google-deepmind/penzai`, 2024–2026.

[5] Stephan Hoyer and Joseph Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 2017.

[6] PyTorch. Named tensors (experimental). `https://pytorch.org/docs/stable/named_tensor.html`, 2019–2026. API marked experimental; broad-broadcast semantics retained for compatibility.

[7] Alex Rogozhnikov. einops: Clear and reliable tensor manipulations with einstein-like notation. https://github.com/arogozhnikov/einops, 2018–2026.

[8] Stanford CRFM and contributors. Haliax: Named tensors for jax. https://github.com/stanford-crfm/haliax, 2023–2026.

[9] tensor project. ADR-0005: Adopt TeX / LyX as a first-class authoring surface. https://github.com/uyuutosa/tensor/blob/main/docs/arc42/09-decisions/0005-adopt-tex-lyx-as-authoring-surface.md, 2026. Schedules the consteval lift of the _tex UDL parser, pending a fixed-size AST representation that escapes constant evaluation.

[10] tensor project. ADR-0014: Dawn-via-vcpkg substrate strategy for the WebGPU kernel backend. https://github.com/uyuutosa/tensor/blob/main/docs/arc42/09-decisions/0014-dawn-via-vcpkg-substrate.md, 2026.

[11] tensor project. ADR-0021: Hexagonal-lite kernel backend port boundary. https://github.com/uyuutosa/tensor/blob/main/docs/arc42/09-decisions/0021-hexagonal-lite-kernel-backend.md, 2026.

[12] tensor project. Phase 7a M1: Named-tensor library landscape comparison study. https://github.com/uyuutosa/tensor/blob/main/docs/research/phase-7a-m1-landscape-comparison.md, 2026.